

ui.input 全面详细阐述

`ui.input` 是 NiceGUI 框架中基于 Quasar `QInput` 组件实现的文本输入组件，支持实时输入监听、自动补全、表单验证、密码隐藏、样式自定义等核心功能，适用于登录表单、搜索框、数据录入等各类文本输入场景。以下从核心特性、使用方法、参数配置、API 详情等方面展开说明。

一、核心特性

- 基础输入能力**：支持文本输入、占位提示、标签显示，支持密码模式（隐藏输入内容）及密码可见性切换。
- 实时与延迟监听**：`on_change` 事件响应每一次按键输入，也可通过 `keydown.enter`（回车确认）、`blur`（失去焦点）事件实现延迟确认输入。
- 灵活验证机制**：支持字典式规则验证、自定义函数验证（含异步验证），可手动触发验证并获取错误信息。
- 自动补全功能**：提供输入建议列表，提升输入效率，支持自定义建议选项。
- 样式高度可定制**：支持 Quasar 原生属性（如圆角、边框、紧凑模式），可通过 `input-class`/`input-style` 自定义输入框样式，支持插槽添加自定义元素（如自定义清除按钮）。
- 组件联动**：支持与标签、开关等组件进行值绑定、启用状态绑定、可见性绑定，实现复杂交互逻辑。

二、基础使用方法

1. 最简示例：实时监听输入

通过 `on_change` 实时响应输入变化，结合标签组件显示输入内容，并添加输入长度验证：

```
from nicegui import ui

# 标签用于显示输入结果
result = ui.label()

# 基础输入框：带标签、占位提示、实时监听、长度验证
ui.input(
    label='文本输入',
    placeholder='开始输入...',
    on_change=lambda e: result.set_text(f'你输入了: {e.value}'), # 实时更新标签
    validation={'输入过长(限20字)': lambda value: len(value) < 20} # 验证规则：长度<20
)

ui.run()
```

2. 密码输入模式

启用 `password` 参数隐藏输入内容，可选 `password_toggle_button` 显示密码切换按钮：

```
from nicegui import ui

# 基础密码输入（无切换按钮）
ui.input(label='密码', password=True, placeholder='输入密码')
```

```
# 带可见性切换的密码输入
ui.input(
    label='带切换密码',
    password=True,
    password_toggle_button=True, # 显示"眼睛"图标, 点击切换可见性
    placeholder='输入密码 (可切换可见)'
)

ui.run()
```

3. 自动补全功能

通过 `autocomplete` 参数传入字符串列表, 输入时显示匹配建议:

```
from nicegui import ui

# 自动补全选项列表
suggestions = ['NiceGUI', 'AutoComplete', 'Python', 'Web开发', '前端框架']

ui.input(
    label='带补全的输入',
    placeholder='输入关键词...',
    autocomplete=suggestions # 绑定补全建议
)

ui.run()
```

4. 可清除输入

通过 Quasar 的 `clearable` 属性添加内置清除按钮, 点击一键清空输入内容:

```
from nicegui import ui

# 可清除输入框
input_box = ui.input(value='默认文本', label='可清除输入').props('clearable')

# 标签实时显示输入值 (双向绑定)
ui.label().bind_text_from(input_box, 'value')

ui.run()
```

5. 输入验证 (两种方式)

支持「自定义函数验证」和「字典规则验证」, 支持异步验证 (2.7.0+ 版本):

```
from nicegui import ui
import asyncio

# 方式1: 自定义函数验证 (返回错误信息或 None)
ui.input(
    label='姓名 (函数验证)',
```

```

        validation=lambda value: '长度至少5个字符' if len(value) < 5 else None
    )

# 方式2: 字典规则验证 (键为错误信息, 值为验证函数, 返回 True 表示有效)
ui.input(
    label='姓名 (字典验证)',
    validation={'长度至少5个字符': lambda value: len(value) >= 5}
)

# 方式3: 异步验证 (2.7.0+)
async def async_validation(value):
    await asyncio.sleep(0.5) # 模拟异步操作 (如接口校验)
    return '已被占用' if value == 'admin' else None

ui.input(
    label='用户名 (异步验证)',
    placeholder='输入用户名 (admin 已被占用)',
    validation=async_validation
)

ui.run()

```

6. 自定义样式

通过 Quasar 属性、`input-class`/`input-style` 或插槽自定义样式:

```

from nicegui import ui

# 1. 使用 Quasar 原生属性 (圆角、边框、紧凑模式)
ui.input(placeholder='圆角+边框+紧凑', props='rounded outlined dense')

# 2. 自定义输入框文本样式 (字体、颜色)
ui.input(
    label='自定义文本样式',
    value='蓝色等宽字体',
    props='input-style="color: blue; font-size: 16px" input-class="font-mono"'
)

# 3. 插槽添加自定义清除按钮
with ui.input(value='自定义清除按钮', label='高级自定义', classes='w-64') as custom_input:
    # 清除按钮: 点击清空值, 仅当有值时显示
    ui.button(
        color='orange-800',
        icon='delete',
        on_click=lambda: custom_input.set_value(None)
    ).props('flat dense').bind_visibility_from(custom_input, 'value')

ui.run()

```

7. 延迟确认输入（回车 / 失去焦点）

若无需实时监听，可通过 `keydown.enter`（回车）或 `blur`（失去焦点）事件触发回调：

```
from nicegui import ui

result = ui.label()

input_box = ui.input(
    label='延迟确认输入',
    placeholder='按回车或失去焦点确认'
)

# 回车触发
input_box.on('keydown.enter', lambda e: result.set_text(f'回车确认: {e.sender.value}'))

# 失去焦点触发
input_box.on('blur', lambda e: result.set_text(f'失去焦点确认: {e.sender.value}'))

ui.run()
```

三、关键参数说明

- `label`：类型为 `str | None`，输入框的显示标签，用于说明输入用途，无默认值。
- `placeholder`：类型为 `str | None`，输入框为空时显示的提示文本，无默认值。
- `value`：类型为 `str | None`，输入框的初始值，默认值为 `None`。
- `password`：类型为 `bool`，是否启用密码模式（隐藏输入内容），默认值为 `False`。
- `password_toggle_button`：类型为 `bool`，是否显示密码可见性切换按钮（仅 `password=True` 时生效），默认值为 `False`。
- `on_change`：类型为 `Callable[[ValueChangeEventArguments], Any] | Callable[[], Any]`，输入值变化时（每一次按键）触发的回调函数，事件对象 `e` 包含 `value` 属性（当前输入值），无默认值。
- `autocomplete`：类型为 `list[str] | None`，自动补全建议列表，输入时匹配显示，默认值为 `None`。
- `validation`：类型为 `dict[str, Callable[[Any], bool]] | Callable[[Any], str | None | Awaitable[Optional[str]]] | None`，输入验证规则。支持字典（错误信息→验证函数）或自定义函数（返回错误信息 / `None`，支持异步），默认值为 `None`（无验证）。

四、高级功能

1. 组件双向绑定

将输入框与其他组件（如滑块、开关）绑定，实现值同步：

```

from nicegui import ui

# 输入框与滑块绑定（同步数值）
input_num = ui.input(label='数值输入', value='50')
slider = ui.slider(min=0, max=100, value=50)

# 双向绑定：输入框修改 → 滑块同步；滑块拖动 → 输入框同步
input_num.bind_value(slider, 'value', forward=lambda x: int(x) if x.isdigit() else 0)
slider.bind_value(input_num, 'value', forward=lambda x: str(x))

ui.run()

```

2. 手动触发验证与禁用自动验证

通过 `validate()` 方法手动触发验证，通过 `without_auto_validation()` 禁用实时自动验证：

```

from nicegui import ui

# 禁用自动验证，仅手动触发
input_box = ui.input(
    label='手动验证',
    validation=lambda value: '长度至少3个字符' if len(value) < 3 else None
).without_auto_validation()

# 验证按钮：点击触发验证
def trigger_validation():
    is_valid = input_box.validate() # 手动触发验证，返回是否有效
    ui.notify(f'验证结果: {"有效" if is_valid else input_box.error}')

ui.button('点击验证', on_click=trigger_validation)

ui.run()

```

3. 动态修改属性（标签、补全列表）

通过 `set_label()`、`set_autocomplete()` 等方法动态修改输入框属性：

```

from nicegui import ui

input_box = ui.input(label='初始标签', placeholder='输入...')

# 动态修改标签
ui.button('修改标签', on_click=lambda: input_box.set_label('修改后的标签'))

# 动态修改自动补全列表
new_suggestions = ['动态补全1', '动态补全2', '动态补全3']
ui.button('更新补全列表', on_click=lambda: input_box.set_autocomplete(new_suggestions))

ui.run()

```

4. 启用 / 禁用与显示 / 隐藏控制

通过 `set_enabled()`、`set_visibility()` 或绑定开关组件控制输入框状态：

```
from nicegui import ui

input_box = ui.input(label='可控制输入框', placeholder='输入内容')

# 开关控制启用/禁用
enable_switch = ui.switch(label='启用输入', value=True)
enable_switch.bind_value_to(input_box, 'enabled')

# 开关控制显示/隐藏
show_switch = ui.switch(label='显示输入框', value=True)
show_switch.bind_value_to(input_box, 'visible')

ui.run()
```

五、API 详情补充

1. 核心属性

属性名	类型	说明
<code>classes</code>	<code>str</code>	组件的 HTML 类名，支持 Tailwind/Quasar 类，用于容器样式自定义
<code>client</code>	<code>Client</code>	组件所属的客户端实例
<code>enabled</code>	<code>BindableProperty</code>	是否启用组件（可绑定，支持动态切换）
<code>error</code>	<code>str None</code>	验证失败后的错误信息（可设置）
<code>html_id</code>	<code>str</code>	组件的 HTML DOM ID（版本 2.16.0+）
<code>label</code>	<code>BindableProperty</code>	输入框标签（可绑定，支持动态修改）
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性（如 <code>clearable</code> 、 <code>rounded</code> ）
<code>style</code>	<code>Style[Self]</code>	组件容器的内联 CSS 样式
<code>validation</code>	<code>Callable dict None</code>	验证规则（可设置，支持动态修改）
<code>value</code>	<code>BindableProperty</code>	输入框当前值（可绑定，支持动态同步）
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（可绑定，支持动态切换）

2. 常用方法

方法名	说明	参数
<code>bind_enabled(target)</code>	双向绑定组件启用状态到目标对象	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>enabled</code> ）
<code>bind_label(target)</code>	双向绑定组件标签到目标对象	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>label</code> ）
<code>bind_value(target)</code>	双向绑定组件值到目标对象	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>value</code> ）
<code>set_autocomplete(list)</code>	动态设置自动补全建议列表	<code>list</code> ：字符串列表（ <code>None</code> 清空）
<code>set_enabled(bool)</code>	启用 / 禁用组件	<code>bool</code> ： <code>True</code> 启用， <code>False</code> 禁用
<code>set_label(str)</code>	动态修改输入框标签	<code>str</code> ：新标签（ <code>None</code> 隐藏标签）
<code>set_value(str)</code>	动态设置输入框值	<code>str</code> ：新输入值
<code>set_visibility(bool)</code>	显示 / 隐藏组件	<code>bool</code> ： <code>True</code> 显示， <code>False</code> 隐藏
<code>validate(return_result=True)</code>	手动触发验证	<code>return_result</code> ：是否返回验证结果（异步验证需设为 <code>False</code> ）；返回值： <code>True</code> 有效 / 错误信息
<code>without_auto_validation()</code>	禁用实时自动验证（仅手动触发）	无参数，返回组件实例（支持链式调用）
<code>tooltip(text)</code>	为组件添加悬浮提示	<code>text</code> ：提示文本

3. 事件说明

- `on_change`：输入值实时变化（每一次按键），事件对象含 `value` 属性。
- `keydown.enter`：按下回车键触发，可通过 `e.sender.value` 获取当前值。
- `blur`：输入框失去焦点时触发，适用于延迟确认输入。
- `click`：点击输入框时触发，可用于初始化操作（如加载数据）。

六、注意事项

1. **样式自定义限制**：`ui.input` 基于 Quasar `QInput`，无法直接样式化外层容器，但可通过 `input-class / input-style` 样式化内部原生输入框，或通过 `props` 应用 Quasar 预定义样式。
2. **异步验证使用**：异步验证函数需返回 `Awaitable[Optional[str]]`，调用 `validate()` 时需设置 `return_result=False`，验证结果在后台异步处理。

3. **自动补全兼容性**: 补全功能依赖浏览器原生自动完成机制, 部分浏览器可能需要启用相关设置才能正常显示。
4. **值绑定类型转换**: 与非文本组件 (如滑块、数字输入框) 绑定时, 需通过 `forward/backward` 函数处理类型转换 (如字符串→整数)。
5. **版本兼容性**: `html_id` 属性需 NiceGUI 2.16.0+, 异步验证需 2.7.0+, `strict` 参数绑定需 3.0.0+, 使用时需注意框架版本。

七、应用场景

1. 登录 / 注册表单: 结合密码模式、验证规则 (如密码长度、用户名格式) 实现安全输入。
2. 搜索功能: 通过自动补全提供搜索建议, 提升用户搜索效率。
3. 数据录入: 支持实时验证 (如输入长度、格式), 减少无效数据提交。
4. 配置面板: 与滑块、开关等组件绑定, 实现参数动态调整与同步显示。
5. 异步校验场景: 如用户名唯一性校验、手机号验证码校验, 通过异步验证函数对接后端接口。

`ui.input` 凭借丰富的功能与灵活的自定义能力, 成为 NiceGUI 中最常用的交互组件之一, 可适配从简单文本输入到复杂表单验证的各类场景, 兼顾易用性与扩展性。